TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

CONSTRAINT SATISFACTION PROBLEMS

Crypto Arithmetic & Map Colouring

Lab 5 Report

Submitted By:

Name: **Aditya Kumar Shah**

Roll No: **080BCT011**

Date: **2083-04-02**

Submitted To:

Department of Electronics
and Computer Engineering
Institute of Engineering,
Pulchowk Campus

Subject: Artificial Intelligence

Contents

1 Objectives

1. To understand how **Constraint Satisfaction Problems (CSPs)** are defined by variables, domains, and constraints.
2. To implement the classic **Crypto Arithmetic** puzzle `SEND + MORE = MONEY` in Prolog using finite-domain constraints.
3. To model the **Australia Map Colouring Problem** as a CSP and obtain a valid 3-colouring.
4. To solve the same map-colouring problem in **Python** using brute-force search.
5. To compare declarative and imperative implementations of the same CSP.

2 Theory

A **Constraint Satisfaction Problem** consists of:

- a set of variables $X = \{x_1, x_2, \ldots, x_n\}$,
- a domain D_i of possible values for each variable x_i,
- a set of constraints C that restrict value combinations.

A solution assigns every variable a value from its domain while satisfying all constraints.

2.1 Crypto Arithmetic

In `SEND + MORE = MONEY`, each letter must be assigned a unique digit so that the arithmetic is correct. With carries $C_1, C_2, C_3, C_4 \in \{0, 1\}$ the column equations are:

$$D + E = Y + 10C_1 \tag{1}$$
$$N + R + C_1 = E + 10C_2 \tag{2}$$
$$E + O + C_2 = N + 10C_3 \tag{3}$$
$$S + M + C_3 = O + 10C_4 \tag{4}$$
$$M = C_4 \tag{5}$$

The leading digits S and M cannot be zero, and all letters must represent distinct digits.

2.2 Map Colouring

The task is to colour a map with the fewest colours such that adjacent regions differ. For the Australia map we use seven regions:

Region	Abbreviation
Western Australia	WA
Northern Territory	NT
Queensland	Q
New South Wales	NSW
Victoria	V
South Australia	SA
Tasmania	T

Domain: $D_i = \{\text{red}, \text{green}, \text{blue}\}$. Constraint: every pair of adjacent regions must have different colours. Tasmania has no mainland neighbours.

3 Methodology

Both problems were solved by first writing their CSP formulation and then implementing them. SWI-Prolog 10.0.2 was used for the logic-programming parts with `library(clpfd)`, and Python 3 was used for the brute-force search. The code was edited in a text editor and run from the terminal.

4 Implementation

4.1 Crypto Arithmetic in Prolog

The file `1.pl` encodes the puzzle with CLPFD constraints.

```
% SEND + MORE = MONEY cryptarithmetic puzzle
% Using clpfd (Constraint Logic Programming over Finite Domains)

:- use_module(library(clpfd)).

solution(S, E, N, D, M, O, R, Y) :-
    Vars = [S, E, N, D, M, O, R, Y],
    Vars ins 0..9,
    % All letters must have distinct values
    all_different(Vars),
    % Leading digits cannot be 0
    S #\= 0,
    M #\= 0,
    % C4 must be 1 (carry from the leftmost column)
    M #= 1,
    % Column arithmetic (right to left)
    D + E #= Y + 10 * C1,
    N + R + C1 #= E + 10 * C2,
    E + O + C2 #= N + 10 * C3,
    S + M + C3 #= O + 10 * C4,
    C4 #= 1,
    % Label (find the solution)
    label(Vars).
```

Listing 1: `1.pl` – SEND + MORE = MONEY

Command and output:

```
$ swipl -q -t "solution(S,E,N,D,M,O,R,Y), writeln([S,E,N,D,M,O,R,Y])." 1.pl
[9,5,6,7,1,0,8,2]
```

Thus, $9567 + 1085 = 10652$.

4.2 Map Colouring in Prolog

The file `map_coloring.pl` uses CLPFD to assign one of three colours to each region.

```
% Australia Map Colouring Problem (Constraint Satisfaction)
% Regions: WA, NT, Q, NSW, V, SA, T
% Domain : {red, green, blue}  (encoded as 1, 2, 3)
% Constraint: adjacent regions must have different colours.

:- use_module(library(clpfd)).

% solve(-Colouring)
% Colouring is a list Region-Colour where Colour is in 1..3.
solution(Colouring) :-
    Colouring = [
        wa-WA, nt-NT, q-Q, nsw-NSW, v-V, sa-SA, t-T
    ],
    Colours = [WA, NT, Q, NSW, V, SA, T],
    Colours ins 1..3,

    % Adjacency constraints: neighbouring regions must differ.
    WA #\= NT, WA #\= SA,
    NT #\= Q,  NT #\= SA,
    Q  #\= NSW, Q  #\= SA,
    NSW #\= V, NSW #\= SA,
    V #\= SA,

    label(Colours).

% colour_name(+Code, -Name)
colour_name(1, red).
colour_name(2, green).
colour_name(3, blue).

% print_solution(+Colouring)
print_solution(Colouring) :-
    write('Australia Map Colouring Solution:'), nl,
    forall(
        member(Region-Code, Colouring),
        ( colour_name(Code, Name),
          format('  ~w = ~w~n', [Region, Name]) )
    ).

% all_solutions(-Solutions)
all_solutions(Solutions) :-
    findall(Colouring, solution(Colouring), Solutions).
```

Listing 2: `map_coloring.pl` – Australia map CSP

Command and output:

```
$ swipl -q map_coloring.pl
Australia Map Colouring Solution:
  wa = red
  nt = green
  q = red
  nsw = green
  v = red
  sa = blue
  t = red
```

4.3 Map Colouring in Python

The file `map_coloring.py` uses a brute-force search over all colour combinations.

```python
# Australia Map Colouring Problem (Constraint Satisfaction)
# Regions: WA, NT, Q, NSW, V, SA, T
# Domain : {red, green, blue}
# Constraint: adjacent regions must have different colours.

from itertools import product

REGIONS = ["WA", "NT", "Q", "NSW", "V", "SA", "T"]
COLOURS = ["red", "green", "blue"]

# Adjacency list (undirected).
ADJACENT = {
    "WA": ["NT", "SA"],
    "NT": ["WA", "Q", "SA"],
    "Q": ["NT", "NSW", "SA"],
    "NSW": ["Q", "V", "SA"],
    "V": ["NSW", "SA"],
    "SA": ["WA", "NT", "Q", "NSW", "V"],
    "T": [],  # Tasmania has no mainland neighbours.
}


def is_valid(assignment):
    """Check whether every adjacency constraint is satisfied.""
       "
    for region, colour in assignment.items():
        for neighbour in ADJACENT[region]:
            if assignment.get(neighbour) == colour:
                return False
    return True


```

```python
def solve():
    """Find the first valid colouring by brute-force search."""
    for combo in product(COLOURS, repeat=len(REGIONS)):
        assignment = dict(zip(REGIONS, combo))
        if is_valid(assignment):
            return assignment
    return None


if __name__ == "__main__":
    solution = solve()
    if solution:
        print("Australia Map Colouring Solution:")
        for region in REGIONS:
            print(f"  {region} = {solution[region]}")
    else:
        print("No valid colouring found.")
```

Listing 3: `map_coloring.py` – Brute-force solver

Command and output:

```
$ python3 map_coloring.py
Australia Map Colouring Solution:
  WA = red
  NT = green
  Q = red
  NSW = green
  V = red
  SA = blue
  T = red
```

5 Results

Table 1: Lab 5 result summary

Task	Language	Solution
Crypto arithmetic	Prolog	$S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2$
Map colouring	Prolog	WA/Q/V/T red; NT/NSW green; SA blue
Map colouring	Python	WA/Q/V/T red; NT/NSW green; SA blue

6 Discussion

The Prolog implementations are concise because constraints are expressed directly. The CLPFD solver handles propagation and search automatically once the programmer de-

clares variables, domains, and constraints. This makes the code easy to verify against the mathematical specification.

The Python version is concise: it generates all possible colourings with `itertools.product` and returns the first valid one. The two approaches show that a CSP can be solved declaratively or imperatively; the choice depends on the tools available and the desired control over the search process.

Tasmania's isolation means it imposes no colour constraints, so any of the three colours is valid. In both implementations it received red, but this is arbitrary—other solutions exist by permuting the colours or changing Tasmania's colour.

7 Conclusion

This lab demonstrated the formulation and solution of constraint satisfaction problems using Prolog's CLPFD library and Python brute-force search. The crypto arithmetic and map-colouring problems were both solved correctly, and the matching results from the Prolog and Python implementations confirm the correctness of the CSP model.